Final Project Design Phase

Computer Science - Week 11

Oct 3, 2026 - Version 0.0.4

For complete instructions and project ideas, refer to the assignment in Canvas. Specifically, review the final project requirements before you continue below. If you have not reviewed the project requirements in Canvas (you might have to go to the modules view and check the current or past week's content), you will not be able to properly complete the parts below. In addition, it may be helpful to review the information on this Drafter documentation page.

0) Project Planning and Preparation

Successful programmers spend time planning before they begin coding. *After* reviewing the project requirements, answer the following questions to help guide your design and development process.

A. What concepts from class or previous assignments might help you complete this project? Which programming concepts (loops, conditionals, lists, classes, etc.) do you expect to use? What programming skills do you want to improve while completing this project?

From the previous assignment, I will use lists, dictionaries, dataclasses, for loops, conditional statements if, elif, else, functions, Drafter routing @route, and session State management.

B. Which parts of the project do you expect to be the easiest for you? The hardest? Why?

Easiest: Defining the data structures (dictionaries and dataclasses)

Hardest: Conditional filtering to ensure the app doesn't crash when user preferences return an empty list.

C. Even though you have not yet designed the final project, estimate which tasks will likely require the most time and effort? Which features or components should be implemented first?

The backend filtering logic and linking form inputs to State will require the most time

and effort.

D. What distractions might interfere with your work and how will you minimize them? What resources will you need available while working? What questions, if any, do you still have about the assignment requirements?

Like studying, setting aside coding blocks with notifications silenced. Some resources I can use are drafter documentation, python reference guides, and AI assistance for debugging stack traces.

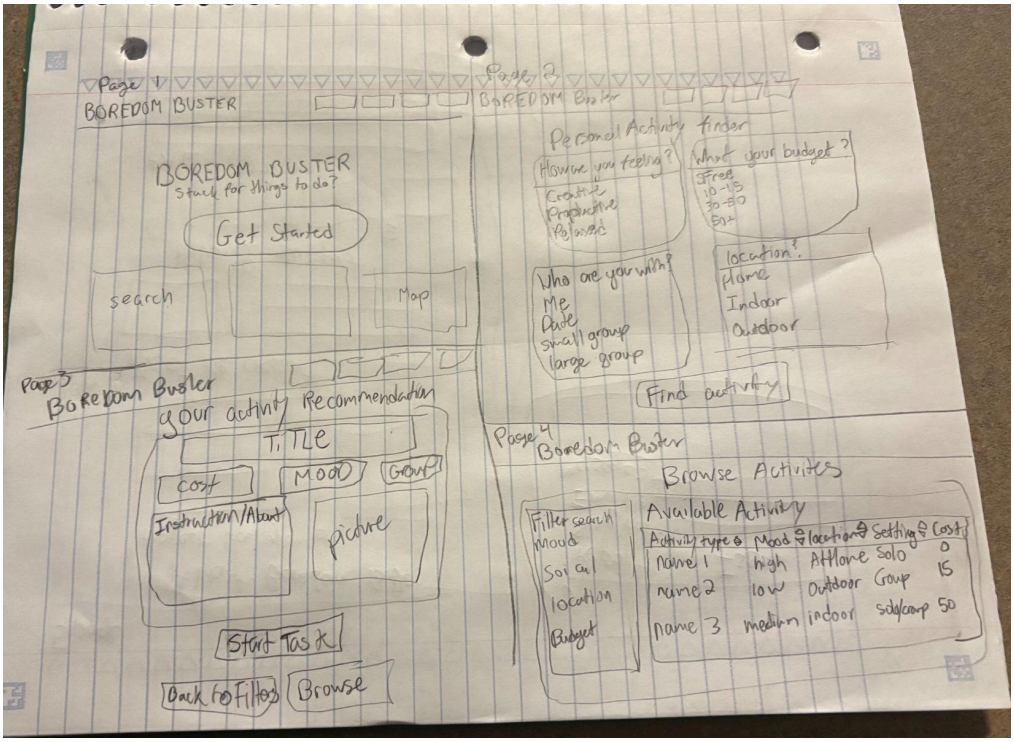
1) Site Diagram

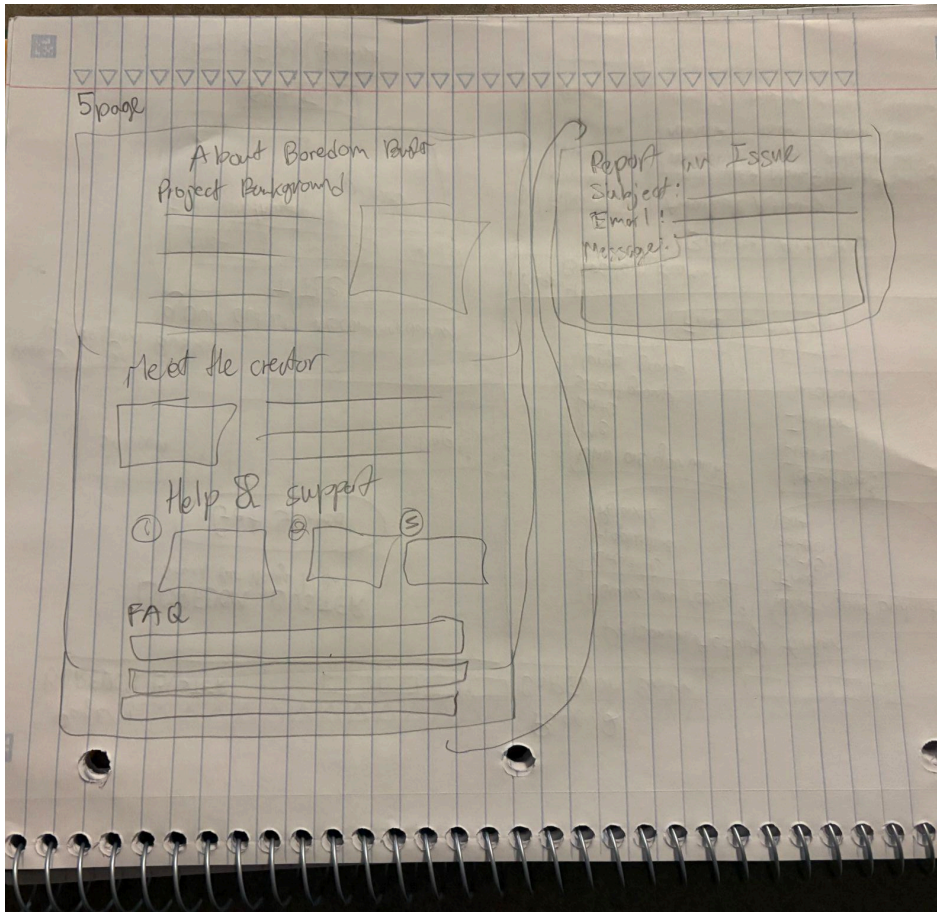
Create a mock-up of your planned website. We suggest you make the mockup on paper, although you can also use software like PowerPoint or Slides to make a mockup. There is other design software that could also be used, but we strongly encourage you to keep it simple. Do not spend more than 15-30 minutes detailing the mock-up. Your goal should be to make a very quick and dirty diagram, to show that you have some kind of idea.

We recommend drawing each page as a rectangle, and laying out the text, buttons, and other components inside. Then, use arrows to indicate links between pages. Label each page the way you will name each function, and then put labels on the arrows to indicate what the button links. Also write down any parameters that should come into the pages.

In addition to the pages and routes, write down what fields your `State` will have, both their names and types. Think clearly about what data will need to be persistent across the entire website.

Add photo(s) or screenshot(s) of your website mock-up and labeled diagram on the next page.


Page 1
BOREDOM BUSTER
BOREDOM BUSTER
Stuck for things to do?
Get Started
search
Map
Page 2
BOREDOM Buster
Personal Activity finder
How are you feeling?
Creative
Productive
Relaxed
What your budget?
Free
10-15
30-50
50+
Who are you with?
Me
Date
small group
large group
location?
Home
Indoor
Outdoor
Find activity
Page 3
Boredom Buster
your activity Recommendation
TITLE
cost
MOOD
Group
Instruction/About
picture
Start Task
Back to Filters
Browse
Page 4
Boredom Buster
Browse Activites
Filter search
Mood
Social
location
Budget
Available Activity
Activity type
Mood
location
Setting
Cost
name 1
high
Athome
Solo
0
name 2
low
Outdoor
Group
15
name 3
medium
indoor
sidgroup
50


5page
About Boredom Buster
Project Background
Report an Issue
Subject:
Email:
Message:
Meet the creator
Help & support
FAQ

2) Implementing Requirements

This table should serve as a draft for how you will implement the requirements into your final project. You may change your ideas later on, but your final site must fulfill all the requirements.

Requirement	**Implementation**
Purpose/Functionality A 1-2 sentence description of the purpose or functionality of your site	A web application that takes user inputs (mood, budget, solo vs. group, location) and dynamically filters a database of activities to present a single recommended task, eliminating decision paralysis.
State Dataclass Attributes Minimum of 4 attributes, one of which must be a list/dict of instances of your second dataclass. Briefly describe each attribute, its type, and how it will be meaningfully modified throughout the site.	1. user_mood(String): Stores selected energy/mood level. 2. user_budget (float): Stores max spending limit. 3. user_location (String): Stores location setting (Home, Indoor, Outdoor). 4. Activities (List[Activity]): Master list of activity instances stored in state and modified when users add new options.
Second Dataclass Your second dataclass, including its attributes, their types, and how they will be meaningfully modified (if they are modified)	**ActivityDataclass:** title (String) mood (String) cost (Float) group_setting (String) location (String) Modification: Instances are iterated over during filtering and appended to when users submit custom activities.

Routes (minimum 7) The names of your route functions	1. @route("/") (Home) 2. @route("/preferences") (Input Form) 3. @route("/results") (Recommendation) 4. @route("/browse") (Browse All) 5. @route("/add") (Add Activity Form) 6. @route("/about") (About Page) 7. @route("/reset") (Reset State)
Pages (minimum 5) A brief description of the pages in your site	1. **Home:** Welcome and introductory overview. 2. **Preferences:** Input form for mood, budget, group setting, and location. 3. **Results:** Displays the randomly selected matching activity card. 4. **Browse:** Full table view of all available activities. 5. **About:** Explains the project's purpose and how it combats option paralysis.
User Input (minimum 3) Description of user input components (TextBox, TextArea, CheckBox, or SelectBox) and their purposes For more information on these input types, see Drafter Documentation.	1. SelectBox for selecting Mood / Energy Level. 2. NumberBox or SelectBox for specifying Maximum Budget. 3. SelectBox for choosing Location Setting (Home, Indoor, Outdoor).
If Statements (minimum 3) Brief description of what will be evaluated in your if statements	1. Evaluates if the filtered list is empty to trigger a fallback recommendation.

and how they will meaningfully change the logic of the site	2. Checks if an activity's cost is less than or equal to the user's maximum budget. 3. Verifies if the activity's mood tag matches the user's selected energy level.
Loop (minimum 1) This loop must iterate over the list/dict of dataclass instances in your State attribute. Description of this loop and why it is meaningful to the site	A for loop that iterates over the activities list in the State attribute to evaluate matching conditions and build the filtered subset.

LLM Conversation

If you used an LLM in any part of this work, copy and paste every part of that conversation below. Otherwise, you may be given a zero for the assignment. Obviously, these words will not all be yours, and this is the exception to the requirement at the top of this assignment that all responses be in your own words. If you did not use an LLM to assist you in this assignment, simply say so below.

User: I am designing my CS1 final project using Drafter. How should I structure my State dataclass, routes, and filtering logic for an anti-boredom decision helper app?

LLM:Designing an anti-boredom decision helper app in **Drafter** is a great CS1 project choice because it lets you combine explicit state management, list filtering, and interactive UI components cleanly.

Here is how you can structure your `State` dataclass, filtering logic, and routes to keep your code organized and easy to test.

1. Structuring the `State` Dataclass

Your `State` dataclass should hold everything that changes or needs to be remembered as the user interacts with the app. For a decision helper, this includes your master list of activities, any active filter settings, and the currently chosen suggestion.

2. Writing the Filtering Logic

Keep your filtering logic separate from your UI route functions. This makes your code much easier to debug and test independently (a common requirement in CS1 grading

rubrics).

3. Setting Up Routes and Event Handlers

In Drafter, routes are plain Python functions decorated with `@route` that take the current `State` and return a `Page`. You will need a main index route and handler functions that update the state when buttons are clicked or forms are submitted.

Tips for Expanding Your Project:

- ***Adding User Inputs:*** *If you want users to type in their own custom activities, you can add text input components and route handlers that append new dictionaries to `state.all_activities`.*
- ***Modular Design:*** *Keep data definitions at the top, helper logic (like `filter_activities`) in the middle, and `@route` UI functions at the bottom.*

Final Submission

When you are finished, download this file as a Word Document (docx) and upload the file to the appropriate assignment on Canvas.